PRE-MIDTERM LECTURE

Transform and Conquer

Change the problem into a form that is easier to solve

CORE IDEA

transform → solve

same goal

easier form

Learning outcomes

RECOGNIZE

Explain when transformation is the central design choice.

Distinguish transformation from decrease and divide.

APPLY

Use presorting and representation changes to simplify common tasks.

REASON

Explain preprocessing trade-offs and simple problem reductions.

Where Transform and Conquer fits

Strategy	What the algorithm does
Brute Force	Try the straightforward possibilities
Decrease & Conquer	Solve ONE smaller instance
Divide & Conquer	Solve MULTIPLE smaller instances
Transform & Conquer	Change the problem into an EASIER form

The core idea

TRANSFORM

Change the instance, representation, or formulation.

CONQUER

Solve the transformed version using a suitable known method.

INTERPRET

Use that solution to answer the original problem.

Original problem → transformed problem → solution

The three forms in our CDF

MORE CONVENIENT INSTANCE

Change the input into a friendlier arrangement.

Classic idea: presorting.

DIFFERENT REPRESENTATION

Store the same information in a structure that makes required operations easier.

PROBLEM REDUCTION

Convert problem A into problem B, where B already has a known solution.

Why transform at all?

DO EXTRA WORK FIRST

Transformation may cost time or memory up front.

MAKE LATER WORK CHEAPER

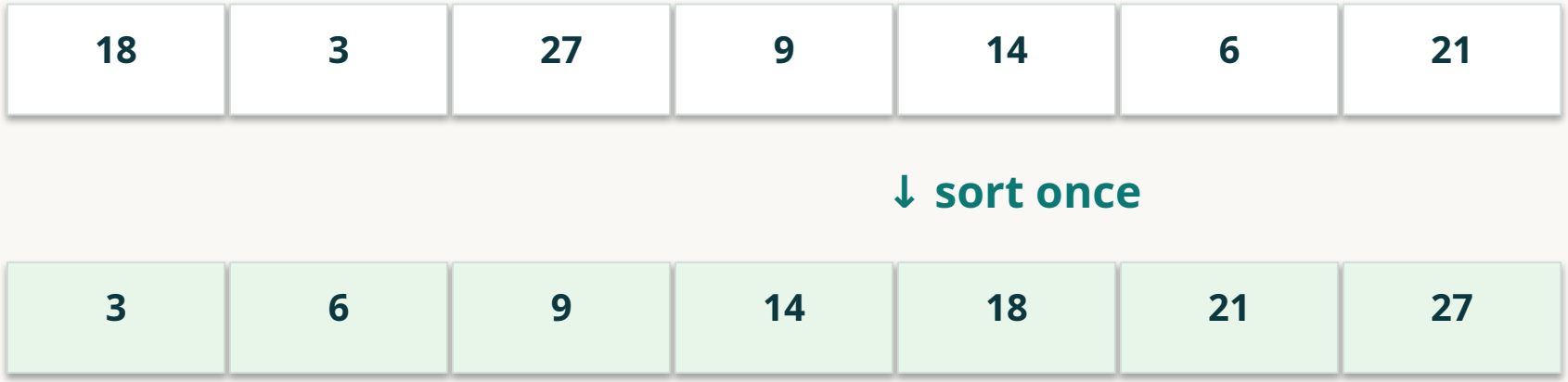
Queries or later algorithmic steps may become much faster.

REUSE THE TRANSFORMED FORM

The up-front cost is especially useful when many operations follow.

Preprocessing is useful only when its benefit justifies its cost.

Form 1 — transform to a more convenient instance



Same values. More useful arrangement.

The transformation does not solve every future task—but it can make many of them easier.

Repeated searching — original form

UNSORTED ARRAY

Each search may need Linear Search.

Worst case per query: Θ(n).

For q searches: Θ(qn).

QUESTION

Would it ever be worth doing extra work before the searches begin?

The input itself has not shrunk. We are considering changing its form.

EXAMPLE

Repeated searching — transformed form

STEP 1 — SORT

Sort once: typically Θ(n log n).

STEP 2 — SEARCH

Use Binary Search: Θ(log n) per query.

TOTAL FOR Q QUERIES

Θ(n log n + q log n).

Transformation cost + cheaper repeated operations

One search vs many searches

Situation	Linear search	Presort + binary search
One search	$\Theta(n)$	$\Theta(n \log n) + \Theta(\log n)$
Many searches	$\Theta(qn)$	$\Theta(n \log n + q \log n)$
Best choice depends on	How many queries?	Whether preprocessing can be reused

A faster query algorithm does not automatically mean a faster total solution.

Presorting can simplify duplicate detection



AFTER SORTING

Equal values become adjacent. One linear scan can detect duplicates.

Presorting can reveal order-based structure

MEDIAN / ORDER

Sorting exposes relative order and positions.

COMMON ELEMENTS

Sort both collections and scan them together.

GROUPING

Equal keys become contiguous, simplifying frequency/group operations.

Transform first when the ordered structure will be useful later.

Transformation is not free

TIME

Sorting, heap construction or conversion can add preprocessing time.

SPACE

A different representation may require additional memory.

REUSE

Transformation pays off when later work is sufficiently cheaper or repeated.

Form 2 — transform to a different representation

SAME INFORMATION

The underlying problem and data do not change.

Only the way the data is represented changes.

DIFFERENT OPERATIONS BECOME EASIER

Choose a representation that matches the operations the algorithm needs to perform efficiently.

Representation is an algorithmic design decision.

Array → Heap

4	10	3	5	1

BUILD-MAX-HEAP

10	5	3	4	1

WHY THE TRANSFORMATION HELPS

The maximum value is now at the root, so repeated maximum extraction becomes structured and efficient.

What Heap Sort was doing conceptually

TRANSFORM

Convert the array into a max-heap.

CONQUER

Repeatedly move the maximum to its final position.

RESTORE

Heapify the remaining active heap after each extraction.

Same algorithm can be classified along several dimensions.

Graph representation changes what is convenient

Representation	Strength	Typical trade-off
Adjacency Matrix	Fast direct edge lookup	Uses $\Theta(V^2)$ space
Adjacency List	Compact for sparse graphs; easy neighbor iteration	Direct edge lookup may require scanning a list

Same graph → different representation → different operation costs.

Choosing a representation is part of design

ASK 1

Which operations will happen most often?

ASK 2

What time and space costs does each representation create?

ASK 3

Is the transformation cost paid once and reused many times?

Form 3 — problem reduction

Problem A → Problem B → known algorithm for B → answer for A

THE IDEA

Do not invent a completely new solution if the problem can be converted into another problem we already know how to solve.

The conversion must preserve the information needed to recover the original answer.

Simple reduction example — LCM through GCD

ORIGINAL PROBLEM

Find LCM(a,b).

We already have an efficient algorithm for GCD: Euclid's algorithm.

REDUCE TO A KNOWN PROBLEM

Compute g = gcd(a,b).

Then LCM(a,b) = |ab| / g.

LCM problem → GCD problem → Euclid → LCM answer

Reduction must preserve correctness

CONVERT

Map the original instance to the new problem.

SOLVE

Use a correct algorithm for the new problem.

RECOVER

Translate the result back to the original question.

A cheap conversion is useful only if solving the reduced problem really answers the original one.

Do not confuse the three forms

Form	What changes?	Example
More convenient instance	Arrangement of the input	Presort before repeated search
Different representation	How the same data is stored	Array ~~h~~eap; matrix ↔️ adjacency list
Problem reduction	The problem being solved	LCM → GCD

Transform vs Decrease vs Divide

Question to ask	Decrease & Conquer	Divide & Conquer	Transform & Conquer
What happens first?	Reduce	Split	Change form
How many smaller instances?	Usually one	Usually multiple	Not the defining feature
Main idea	Solve a smaller instance	Combine subproblem solutions	Make the problem easier
Example	Binary Search	Merge Sort	Presorting / Heap transformation

A classification question

SCENARIO

You have 100,000 unsorted records and expect 50,000 searches. You sort once, then use Binary Search for every query.

NOT BRUTE FORCE

The strategy deliberately preprocesses the data.

NOT PRIMARILY DECREASE

Binary Search is decrease-and-conquer, but the overall design begins with transformation.

TRANSFORM + CONQUER

Presorting changes the instance so repeated searching becomes cheaper.

Another classification question

SCENARIO

An array is rearranged into a max-heap before repeated maximum removals.

ANSWER

The initial conversion is a transformation to a different representation.

Heap Sort then repeatedly uses the heap property to finish the sorting process.

Common mistakes

MISTAKE 1

“If the input changes, it must be Transform and Conquer.”

No. The change must be the central design strategy that makes the problem easier.

MISTAKE 2

Ignoring transformation cost.

Preprocessing must be included in total analysis.

MISTAKE 3

Thinking classifications are mutually exclusive.

One algorithm can be in-place, comparison-based and transform-oriented at the same time.

Practice — identify the strategy

1. Sort a list once, then answer 10,000 membership queries using Binary Search.
2. Split an array into two halves, recursively sort both halves, then merge.
3. Search only one half of a sorted array after each comparison.
4. Convert an array into a max-heap before repeatedly extracting the maximum.
5. Compute LCM using a GCD algorithm and a simple formula.

Practice answers

Item	Main strategy
1	Transform and Conquer — convenient instance / presorting
2	Divide and Conquer
3	Decrease and Conquer by a constant factor
4	Transform and Conquer — different representation
5	Transform and Conquer — problem reduction

Practice — total-cost reasoning

QUESTION

For n records and q membership queries, compare repeated Linear Search with “sort once + Binary Search.”

Repeated Linear Search: Θ(qn)

Presort + Binary Search: Θ(n log n + q log n)

The better choice depends on q, whether the data changes, and whether the preprocessing can be reused.

Pre-midterm synthesis

Paradigm	Core question
Brute Force	Can I systematically try the straightforward possibilities?
Decrease & Conquer	Can I solve one smaller instance?
Divide & Conquer	Can I split into multiple smaller instances and combine?
Transform & Conquer	Can I change the problem into an easier form?

What should you be able to do in the midterm?

RECOGNIZE

Identify which design strategy an algorithm is using and justify your answer.

TRACE

Execute representative algorithms correctly on a small input.

ANALYZE

Write a recurrence or operation count and derive the appropriate complexity.

Do not memorize labels — explain what the algorithm actually does.

Transform and Conquer — recap

Form	Memory hook
More convenient instance	Rearrange the input so later work is easier
Different representation	Store the same information in a better structure
Problem reduction	Convert to another problem you already know how to solve

Transformation is worthwhile when the cost of changing form is repaid by simpler or repeated work.